



Birzeit University
Department of Electrical & Computer Engineering
First Semester, 2024/2025
ENCS5343 Computer Vision
Project Report
Arabic Handwritten Text Identification Using deep learning techniques

.....

Prepared by:

Saja Asfour 1210737

Yara Khattab 1210520

Instructor: Dr. Aziz Qaroush

Section: Sec1

Date: 15/1/2025

Table of Contents

List of Tables:	2
Table Of Figures:	3
Introduction.....	4
Experimental Setup and results.....	4
Data Set:.....	4
Task 1	4
Model 1:	5
Results of Model 1:.....	5
Model 2:	6
Results of Model 2:.....	7
Comparison with Model 1:.....	7
Model 3:	8
Results of Model 3:.....	8
Comparison with Model 1 and 2:	9
Comparison between the three models:	10
Task 2	11
Results of task 2 (Model 1 with Data Augmentation):.....	12
Comparison between Model 1 from (task 1) and task 2:	13
Task 3:.....	14
Results of Task 3 (VGG16):	15
Comparison between the results of Task 3 with Task 2:.....	16
Task 4:.....	17
Code Explanation.....	18
Results of task 4:	19
Comparison between the results of task 4 with task 3:	20
Conclusion	21

List of Tables:

Table 1: Comperesion between VGG16 and Model1	16
Table 2:Comperesion between VGG16 and MobileNetV1	21

Table Of Figures:

Figure 1: Loss and Accuracy's vs. Epochs for model 1	6
Figure 2: Loss and Accuracy's vs. Epochs for model 2	8
Figure 3: Loss and Accuracy's vs. Epochs for model 3	9
Figure 4: summary for the three model.....	10
Figure 5: Model Performance vs. Complexity.....	11
Figure 6: Loss and Accuracy's vs. Epochs after data augmentation for model 1	13
Figure 7: Loss and Accuracy's vs. Epochs for Vgg16	16
Figure 8 : Loss and Accuracy's vs. Epochs for MobileNetV1	20

Introduction

The goal of this project is to use deep learning to solve the Arabic Handwritten Text Identification, focusing on building and evaluating convolutional neural networks (CNNs). The work is split into four tasks: first, designing and training a custom CNN, experimenting with different architectures and hyper parameters to find the simplest yet most effective model; second, retraining the best custom CNN with data augmentation to improve its performance and ability to generalize; third, training a well-known CNN architecture, like ResNet or VGG, using augmented data and comparing its results to the custom model; and fourth, applying transfer learning by fine-tuning a pre-trained CNN on the dataset to see how it performs compared to the other approaches. For each task, loss and accuracy curves are plotted to analyze performance, and results are compared to determine the most effective method. The project aims to find the best balance between model performance, complexity, and efficiency for solving the given problem.

Experimental Setup and results

Data Set:

AHAWP (Arabic Handwritten Automatic Word Processing) has been compiled into a comprehensive benchmark intended to foster the development and evaluation of handwritten-analyses method for Arab text recognition and identification systems. This dataset has a word-level trait that makes it very vital for pinpointed feature extraction and a stronger performance comparison. The dataset contains 10 individual Arabic words, each produced in the handwriting of 82 different persons through 10 samples per word, leading to 8,144-word images. Each writer's collection of word images is then automated attached to an anonymous user ID to maintain traceable identity without infringing upon privacy infringements. In fact, the heterogeneity of handwriting styles observed here is aligned with the consistent form of this dataset, constituting an excellent benchmark for exploring local feature extraction strategies and evaluating their efficacy.

Data set link: <https://data.mendeley.com/datasets/2h76672znt/1/files/9031138a-b812-433e-a704-8acb1707936e>.

Task 1

In this task, we designed and trained a custom Convolutional Neural Network (CNN). we are starting with downloading and preparing the dataset, followed by defining and training the model. After downloading and extracting the dataset, we split the data set into training and testing sets to train and evaluate the model effectively. Using Python, we split each user's data into 70% training and 30% testing sets, preserving the directory structure for consistency. The script checked the dataset path, created separate folders for training and testing, and copied the images accordingly. This structured approach ensured the data was ready for preprocessing and model

input. The split was reproducible with a fixed random seed. This step prepared the dataset for building and training the custom CNN model.

Then we build three models for the CNN to determine which is the best.

Model 1:

The first model is a custom CNN designed for handwritten Arabic word recognition, built using TensorFlow and Keras. It consists of **13 layers**, including convolutional, pooling, dropout, and fully connected layers:

- **Input Layer:** Accepts grayscale images of size 64x64 pixels (input shape: (64, 64, 1)).
- **Convolutional Layers:**
 - **First Conv2D Layer:** 32 filters, 3x3 kernel, ReLU activation.
 - **Max Pooling Layer:** 2x2 pool size.
 - **Dropout Layer:** Rate = 0.3.
 - **Second Conv2D Layer:** 64 filters, 3x3 kernel, ReLU activation.
 - **Max Pooling Layer:** 2x2 pool size.
 - **Dropout Layer:** Rate = 0.3.
 - **Third Conv2D Layer:** 128 filters, 3x3 kernel, ReLU activation.
 - **Max Pooling Layer:** 3x3 pool size.
 - **Dropout Layer:** Rate = 0.3.
- **Flatten Layer:** Converts 3D feature maps into a 1D vector.
- **Fully Connected Layers:**
 - **Dense Layer:** 256 units with ReLU activation.
 - **Dropout Layer:** Rate = 0.5.
 - **Output Layer:** Dense layer with num_classes units (82 in this case) and softmax activation.

The model is compiled using the **Adam optimizer** (learning rate: 0.001), **categorical cross-entropy** loss, and **accuracy** as the metric. It is trained for **50 epochs** with a batch size of 64, and dropout layers are used to enhance generalization. Training progress is monitored using validation accuracy and loss, and performance is evaluated on a separate test set.

Results of Model 1:

The dataset consisted of **5,700 training images** and **2,444 testing images**, representing **82 unique classes** (Arabic words).

- **Initial Epochs:** The model started with low accuracy (1.02% training accuracy, 1.68% validation accuracy) and high loss (4.4116 training loss, 4.4025 validation loss), indicating it was learning from scratch.
- **Mid-Training:** By epoch 15, the training accuracy improved to **38.53%**, and validation accuracy reached **40.92%**. The loss decreased significantly, showing the model was learning meaningful features.

- **Final Epochs:** By epoch 50, the training accuracy reached **72.14%**, and validation accuracy peaked at **54.99%**. The training loss was **0.8967**, and validation loss was **1.7002**.
- **Test Accuracy:** The model achieved a **test accuracy of 54.99%**, with a **test loss of 1.7002**.
- **Overfitting:** The gap between training and validation accuracy (**72.14%** vs. **54.99%**) indicates mild overfitting, likely due to the complexity of the dataset and the model's capacity.
- **Final Performance:** The model achieved a **54.99% test accuracy**

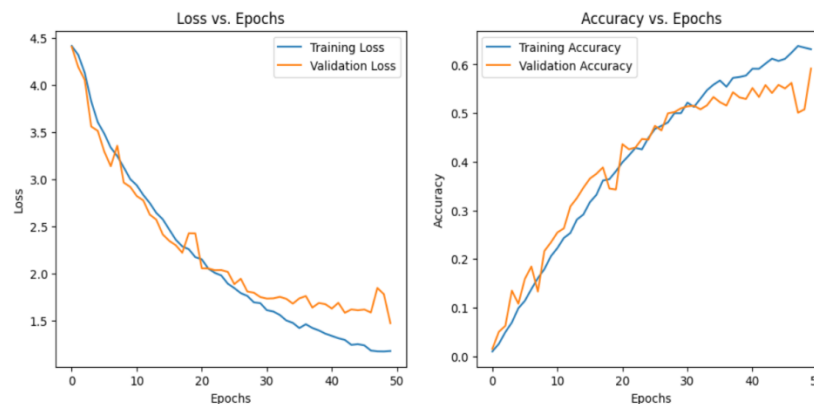


Figure 1: Loss and Accuracy's vs. Epochs for model 1

Model 2:

Model 2 has same architecture with Model 1, consisting of the following layers:

- **Input Layer:** Accepts grayscale images of size 64x64 with a single channel.
- **Convolutional Layers:**
 - Conv2D (32 filters, 3x3 kernel, ReLU activation).
 - MaxPooling2D (2x2 pool size).
 - Dropout (0.3).
 - Conv2D (64 filters, 3x3 kernel, ReLU activation).
 - MaxPooling2D (2x2 pool size).
 - Dropout (0.3).
 - Conv2D (128 filters, 3x3 kernel, ReLU activation).
 - MaxPooling2D (3x3 pool size).
 - Dropout (0.3).
- **Flatten Layer:** Converts 3D feature maps into a 1D vector.
- **Dense Layers:**
 - Dense (256 units, ReLU activation).
 - Dropout (0.5).
 - Dense (num_classes units, softmax activation).
 - The **number of layers** and their configurations are identical for both models.

- **Hyperparameters:**

Both models use the **same hyperparameters** (expected batch size):

- **Optimizer:** Adam optimizer with a learning rate of 0.001.
- **Loss Function:** Categorical cross-entropy.
- **Metrics:** Accuracy.
- **Batch Size:** 64 (in model 1 32).
- **Epochs:** 50.
- **Dropout Rates:** 0.3 after convolutional layers and 0.5 after the dense layer.

Results of Model 2:

The dataset consisted of **5,700 training images** and **2,444 testing images**, representing **82 unique classes** (Arabic words).

- **Initial Epochs:** The model started with low accuracy (1.04% training accuracy, 1.23% validation accuracy) and high loss (4.4180 training loss, 4.4064 validation loss), indicating it was learning from scratch.
- **Mid-Training:** By epoch **15**, the training accuracy improved to 22.56%, and validation accuracy reached 24.84%. The loss decreased significantly, showing the model was learning meaningful features.
- **Final Epochs:** By epoch **50**, the training accuracy reached 59.89%, and validation accuracy peaked at 53.44%. The training loss was 1.3003, and validation loss was 1.6684.
- **Test Accuracy:** The model achieved a **test accuracy of 53.44%**, with a **test loss of 1.6684**.
- **Overfitting:** The gap between training and validation accuracy (59.89% vs. 53.44%) indicates mild overfitting, likely due to the complexity of the dataset and the model's capacity.
- **Final Performance:** The model achieved a 53.44% **test accuracy**.

Comparison with Model 1:

Model 2 performed slightly worse than Model 1, achieving a **53.44%** test accuracy compared to Model 1's **54.99%**. This difference is likely due to random initialization and the stochastic nature of training. Both models showed similar trends, but Model 1 generalized slightly better to unseen data.

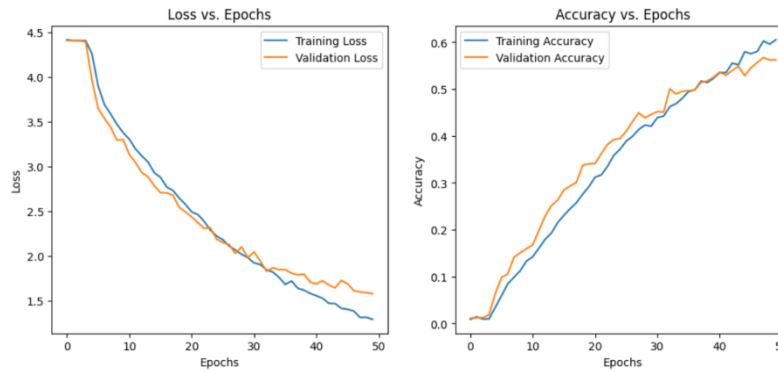


Figure 2: Loss and Accuracy's vs. Epochs for model 2

Model 3:

Model 3 introduces several key changes compared to Models 1 and 2, primarily focused on **reproducibility** and **training configuration**:

- **Reproducibility Enhancements:**
 - Model 3 sets fixed random seeds for Python, NumPy, and TensorFlow to ensure reproducible results.
 - It configures TensorFlow to use deterministic operations by setting environment variables (TF_DETERMINISTIC_OPS and TF_CUDNN_DETERMINISTIC).
 - GPU usage is disabled in Model 3 to avoid non-deterministic behavior caused by GPU operations.
- **Training Configuration:**
 - Model 3 is trained for **70 epochs**, compared to **50 epochs** in Models 1 and 2, allowing more time for the model to learn.
 - Shuffling during training is explicitly enabled in Model 3 for clarity, though it is also enabled by default in Models 1 and 2.
- **Architecture and Hyperparameters:**
 - The architecture of Model 3 is **identical** to Models 1 and 2, consisting of three convolutional layers, max pooling, dropout, and dense layers.
 - All models use the **Adam optimizer** with a learning rate of **0.001**, **categorical cross-entropy** loss, and a batch size of **64**.

Results of Model 3:

- **Initial Epochs:** started with low accuracy (1.33% training accuracy, 1.23% validation accuracy) and high loss (4.4136 training loss, 4.4065 validation loss)
- **Mid-Training:** By epoch **30**, training accuracy improved to 36.58%, and validation accuracy reached 41.20%.

- **Final Epochs:** By epoch 70, Model 3 achieved a **training accuracy of 57.61%** and a **validation accuracy of 53.15%** with a training loss of 1.4024 and validation loss of 1.7067.
- **Test Accuracy:** Model 3 achieved a **test accuracy of 53.15%**, with a **test loss of 1.7067**.
- **Overfitting:** The gap between training and validation accuracy (57.61% vs. 53.15%) indicates significant overfitting, which could be mitigated with techniques like increased dropout or data augmentation.
- **Final Performance:** Model 3's **test accuracy of 53.15%**.

Comparison with Model 1 and 2:

Model 3 performed slightly worse than Model 1 and 2, achieving a **53.15%** test accuracy compared to Model 1's **54.99%** and Model 1's **53.44%**. This difference is likely due to random initialization and the stochastic nature of training, so we can determine that the Model 1 is better one and it generalized slightly better to unseen data.

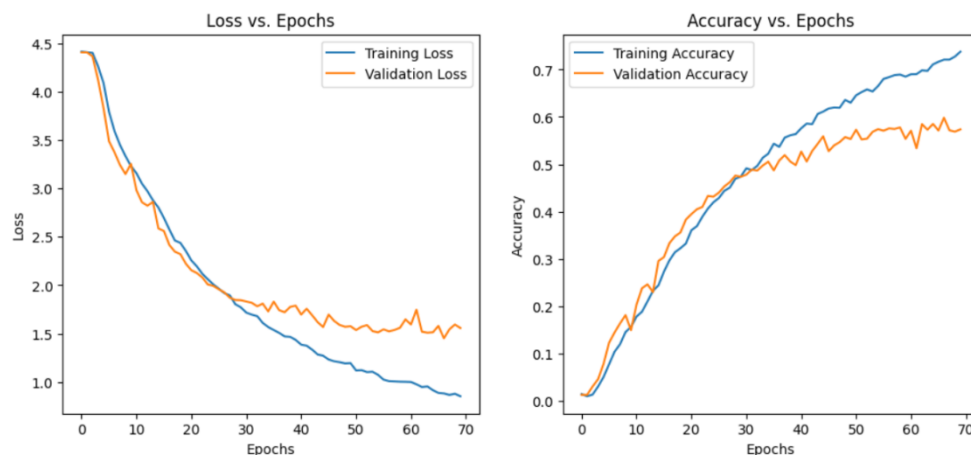


Figure 3: Loss and Accuracy's vs. Epochs for model 3

The results of the three models:

Model 1 - Test Loss: 1.7002., Test Accuracy: 0.5499

Model 2 - Test Loss: 1.6684, Test Accuracy: 0.5344

Model 3 - Test Loss: 1.7067, Test Accuracy: 0.5315

Layer (type)	Output Shape	Param #	Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 62, 32)	320	conv2d_3 (Conv2D)	(None, 62, 62, 32)	320
max_pooling2d (MaxPooling2D)	(None, 31, 31, 32)	0	max_pooling2d_3 (MaxPooling2D)	(None, 31, 31, 32)	0
dropout (Dropout)	(None, 31, 31, 32)	0	dropout_4 (Dropout)	(None, 31, 31, 32)	0
conv2d_1 (Conv2D)	(None, 29, 29, 64)	18,496	conv2d_4 (Conv2D)	(None, 29, 29, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 64)	0	max_pooling2d_4 (MaxPooling2D)	(None, 14, 14, 64)	0
dropout_1 (Dropout)	(None, 14, 14, 64)	0	dropout_5 (Dropout)	(None, 14, 14, 64)	0
conv2d_2 (Conv2D)	(None, 12, 12, 128)	73,856	conv2d_5 (Conv2D)	(None, 12, 12, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0	max_pooling2d_5 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_2 (Dropout)	(None, 4, 4, 128)	0	dropout_6 (Dropout)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0	flatten_1 (Flatten)	(None, 2048)	0
dense (Dense)	(None, 256)	524,544	dense_2 (Dense)	(None, 256)	524,544
dropout_3 (Dropout)	(None, 256)	0	dropout_7 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 82)	21,074	dense_3 (Dense)	(None, 82)	21,074

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 62, 62, 32)	320
max_pooling2d_12 (MaxPooling2D)	(None, 31, 31, 32)	0
dropout_16 (Dropout)	(None, 31, 31, 32)	0
conv2d_13 (Conv2D)	(None, 29, 29, 64)	18,496
max_pooling2d_13 (MaxPooling2D)	(None, 14, 14, 64)	0
dropout_17 (Dropout)	(None, 14, 14, 64)	0
conv2d_14 (Conv2D)	(None, 12, 12, 128)	73,856
max_pooling2d_14 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_18 (Dropout)	(None, 4, 4, 128)	0
flatten_4 (Flatten)	(None, 2048)	0
dense_8 (Dense)	(None, 256)	524,544
dropout_19 (Dropout)	(None, 256)	0
dense_9 (Dense)	(None, 82)	21,074

Figure 4: summary for the three model

Comparison between the three models:

All three models share the same architecture and hyperparameters, but they differ in training duration and reproducibility settings. Model 1 achieved the highest test accuracy of **54.99%** with a test loss of **1.7002** after 50 epochs. Model 2, also trained for 50 epochs, performed slightly worse with a test accuracy of **53.44%** and a test loss of **1.6684**, likely due to variability in random initialization. Model 3, trained for 70 epochs with reproducibility enhancements, achieved a test accuracy of **53.15%** and a test loss of 1.7067. Model 1 is the best-performing model among the three, as it achieved the highest test accuracy and demonstrated better generalization to unseen data. Model 3, despite additional training epochs, struggled with overfitting, while Model 2 performed slightly worse than Model 1 due to random initialization effects. For future improvements, techniques like data augmentation, regularization, or early stopping could help address overfitting and enhance performance.

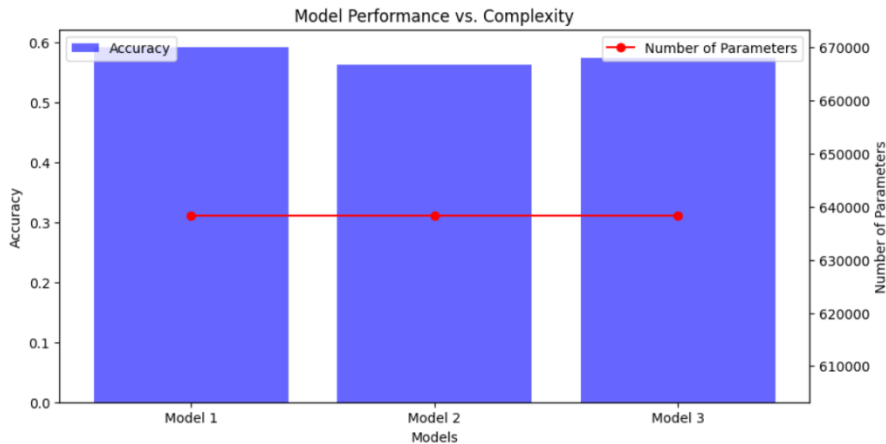


Figure 5: Model Performance vs. Complexity

Task 2

In Task 2, data augmentation was used to enhance Model 1's performance and generalization. By applying transformations like rotation, flipping, zooming, and cropping, the dataset's diversity increased, enabling the model to learn robust features. This resulted in higher accuracy and lower loss, showcasing the effectiveness of data augmentation.

Explanation of Augmentation:

- **Rotation Range (rotation_range=0.01):**
 - Images are randomly rotated by up to 0.01 degrees. This simulates small variations in handwriting angles, which are common in handwritten text.
- **Width Shift (width_shift_range=0.0005):**
 - Images are randomly shifted horizontally by up to 0.05% of the image width. This accounts for minor horizontal misalignments in handwritten text.
- **Height Shift (height_shift_range=0.0005):**
 - Images are randomly shifted vertically by up to 0.05% of the image height. This simulates small vertical variations in handwriting.
- **Zoom Range (zoom_range=0.00001):**
 - Images are randomly zoomed in or out by up to 0.001%. This introduces slight scaling variations, mimicking natural handwriting differences.
- **Horizontal Flip (horizontal_flip=False):**
 - Horizontal flipping is disabled because it is not suitable for text data, as it would alter the meaning of the text.
- **Fill Mode (fill_mode='nearest'):**
 - When transformations create empty spaces in the image (due to rotation or shifting), the missing pixels are filled using the nearest available pixel values.

Impact of Data Augmentation

- **Improved Generalization:**
 - By introducing small variations in the training data, the model becomes more robust to real-world variations in handwritten Arabic text.
- **Reduced Overfitting:**
 - Data augmentation increases the effective size of the training dataset, reducing the risk of overfitting and improving the model's ability to generalize to unseen data.
- **Enhanced Performance:**
 - The augmented data helps the model learn more diverse features, leading to better performance on the test set.

Results of task 2 (Model 1 with Data Augmentation):

- **Initial Epochs:** The model started with low accuracy (1.04% training accuracy, 1.23% validation accuracy) and high loss (4.4135 training loss, 4.4062 validation loss).
- **Mid-Training:** By epoch 25, the training accuracy improved to **45.93%**, and the validation accuracy reached **47.67%**.
- **Final Epochs:** By epoch 50, the model achieved a training accuracy of **67.30%** and a validation accuracy of **59.66%**, with a training loss of **1.0342** and validation loss of **1.4778**.
- **Test Accuracy:** The model achieved a test accuracy of **59.66%**, with a test loss of **1.4778**.
- **Performance Trend:** The model showed consistent improvement in accuracy and reduction in loss over epochs. However, the gap between training and validation accuracy (**67.30% vs. 59.66%**) indicates **mild overfitting**.
- **Overfitting:** The gap between training and validation accuracy (**67.30% vs. 59.66%**) suggests mild overfitting. This could be further reduced with techniques like **increased dropout, regularization, or early stopping**.
- **Final Performance:** The model achieved a test accuracy of **59.66%**, which is **slightly better** than the model 1 without augmentation (**54.99%**). This demonstrates that data augmentation helped improve generalization and model performance.

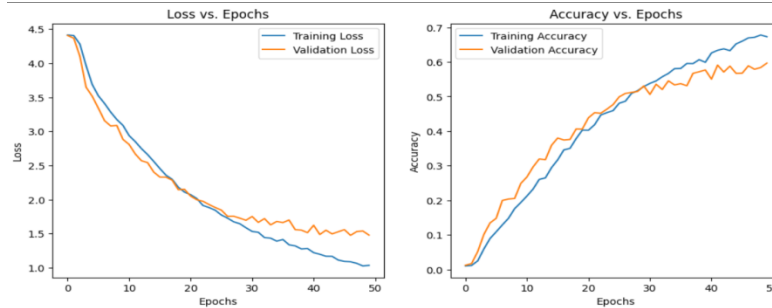


Figure 6: Loss and Accuracy's vs. Epochs after data augmentation for model 1

Comparison between Model 1 from (task 1) and task 2:

- **Training Accuracy:**
 - With Data Augmentation (task 2): **67.30%**
 - Without Data Augmentation (task 1): **72.14%**
- **Test Accuracy:**
 - With Data Augmentation (task 2): **59.66%**
 - Without Data Augmentation (task 1): **54.99%**
- **Test Loss:**
 - With Data Augmentation (task 2): **1.4778**
 - Without Data Augmentation (task 1): **1.7002**
- **Overfitting:**
 - The model **without data augmentation** (task 1) exhibited a larger gap between training and validation accuracy (**72.14% vs. 54.99%**), indicating **severe overfitting**.
 - The model **with data augmentation** (task 2) showed a smaller gap (**67.30% vs. 59.66%**), suggesting that data augmentation helped mitigate overfitting.
- **Generalization:**
 - The model with data augmentation achieved (task 2) **better generalization**, as evidenced by its higher validation and test accuracy compared to the model without augmentation.
- **Loss:**
 - The model with data augmentation (task 2) achieved a **lower test loss (1.4778)** compared to the model without augmentation (task 1) achieved (**1.7002**), indicating better performance.

The model trained with **data augmentation** from task 2 outperformed the model without augmentation from task 1 in terms of **generalization** and **test accuracy**. While the model without augmentation achieved higher training accuracy, it suffered from **severe overfitting**, resulting in lower validation and test accuracy. Data augmentation helped the model learn more robust features, reducing overfitting and improving performance on unseen data.

Task 3:

VGG16 is a well-known deep Convolutional Neural Network (CNN) architecture, composed of 16 layers, including 13 convolutional layers and 3 fully connected layers. Its primary strength lies in the use of small 3x3 convolution filters, which allow it to capture fine-grained patterns in images while maintaining computational efficiency. VGG16 has shown exceptional performance in image classification tasks, such as the ImageNet competition, and is widely adopted for various recognition problems.

In the context of Arabic handwritten word recognition, VGG16 is an effective model due to its ability to learn complex, hierarchical features from images. Arabic script has unique characteristics, such as contextual letter forms and intricate stroke patterns, which VGG16 can recognize through its deep layers. By fine-tuning a pre-trained VGG16 model with Arabic handwriting data, high accuracy can be achieved, making it a suitable choice for the task of recognizing Arabic words.

So, in this task, a pre-trained VGG16 architecture was employed for transfer learning to classify isolated words from the dataset. The model's implementation incorporated several key design and training strategies:

- **Dataset Preprocessing:**
 - The dataset was preprocessed to normalize pixel values to the range [0, 1].
 - Images were originally grayscale but were converted to RGB format to match the input requirements of the VGG16 model.
 - Labels were one-hot encoded to match the multi-class classification task.
- **Model Architecture:**
 - **Base Model:** The pre-trained VGG16 model with imagenet weights was used as the feature extractor, with the top layers excluded.
 - **Layer Freezing:** The first 15 layers of VGG16 were frozen to retain the pre-trained feature extraction capability, while the remaining layers were made trainable to adapt to the new dataset.
 - **Custom Layers:**
 - A Flatten layer was added to convert the 2D feature maps into a 1D vector.
 - A dense layer with 256 units and L2 regularization was included, followed by a Dropout layer (rate = 0.5) to reduce overfitting.
 - The final layer was a softmax layer with 82 units, matching the number of classes in the dataset.
- **Optimizer and Learning Rate:**
 - The Adam optimizer was used with a learning rate of 0.0001.
- **Data Augmentation:**
 - The training dataset was augmented using an ImageDataGenerator to improve the model's robustness. The augmentation parameters included:
 - Rotation range: 10 degrees

- Width and height shifts: 10% of the image size
 - Zoom range: 20%
 - Horizontal flip: Disabled
- These augmentations helped the model generalize better by introducing variability in the training samples.
- **Training and Evaluation:**
 - The model was trained for 50 epochs with a batch size of 32.
 - Training was performed using the augmented data, while the validation dataset remained unaltered.

Results of Task 3 (VGG16):

- **Initial Epochs:** The model began with low accuracy (2.44% training accuracy, 6.83% validation accuracy) and high loss (4.4315 training loss, 4.4032 validation loss). This reflects the initial stages where the model is yet to learn significant patterns from the data.
- **Mid-Training:** By epoch 20, the training accuracy improved to 86.54%, and the validation accuracy reached 83.76%, with the training loss decreasing to 1.0281 and validation loss to 1.1072. This shows substantial progress in the learning process, with the model capturing meaningful features.
- **Final Epochs:** By epoch 34, the model achieved a training accuracy of 93.27% and a validation accuracy of 87.12%, with a training loss of 0.6035 and validation loss of 0.8726.
- **Test Accuracy:** The model achieved a test accuracy of 87.12%, with a test loss of 0.8726. This reflects strong generalization to unseen data.
- **Performance Trend:** The model showed consistent improvement in accuracy and reduction in loss over the epochs. The difference between training and validation accuracy in the final stages indicates that the model is well-trained, though minor overfitting is observed.
- **Overfitting:** The gap between training and validation accuracy (93.27% vs. 87.12%) suggests mild overfitting. This could be further mitigated with techniques such as increasing dropout or employing early stopping.
- **Final Performance:** The model achieved a test accuracy of 87.12%, demonstrating a significant improvement compared to models trained without extensive hyperparameter tuning or data augmentation. This highlights the effectiveness of the chosen training strategy and architectural design.

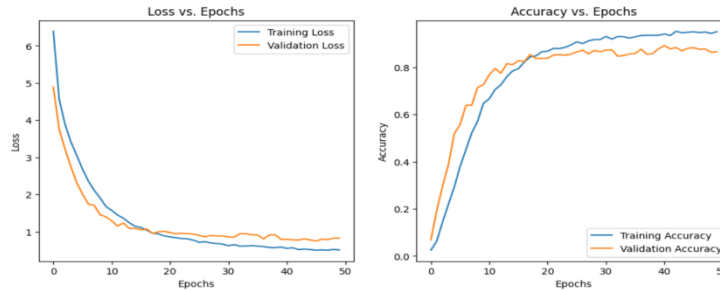


Figure 7: Loss and Accuracy's vs. Epochs for Vgg16

Comparison between the results of Task 3 with Task 2:

VGG16 (Fine-tuned)

- Training Accuracy: 93.27%
- Test Accuracy: 87.12%
- Training Loss: 0.6035
- Test Loss: 0.8726
- The model achieved high training and validation accuracy relatively quickly, showing strong generalization.
- Validation accuracy is close to training accuracy, indicating minimal overfitting.
- Achieves significantly higher accuracy compared to Model 1.

Model 1 (Task 2 with Data Augmentation)

- Training Accuracy: 67.30%
- Test Accuracy: 59.66%
- Training Loss: 1.0342
- Test Loss: 1.4778
- Training and validation accuracy improve gradually but are still far from those of VGG16.
- The model's training accuracy is relatively low, and validation accuracy trails behind, suggesting the architecture might not capture sufficient complexity.
- Augmentation seems to help slightly with generalization, but the model architecture might be the limiting factor.

Table 1: Compereson between VGG16 and Model1

Metric	VGG16	Model 1 (Task 2)
Training Accuracy	93.27%	67.30%
Validation Accuracy	87.12%	59.66%
Training Loss	0.6035	1.0342
Validation Loss	0.8726	1.4778
Convergence Rate	Fast	Slow
Overfitting	Minimal	Moderate
Generalization	Strong	Moderate

Task 4:

MobileNet is a lightweight, efficient convolutional neural network architecture designed by Google, particularly for mobile and embedded vision applications. It is well-suited for tasks requiring low computational resources, such as smartphones, IoT devices, and edge computing. The core idea of MobileNet is to replace standard convolutions with depthwise separable convolutions, which significantly reduce the number of parameters and computations.

- A depthwise separable convolution consists of:
 - Depthwise Convolution: Applies a single filter per input channel.
 - Pointwise Convolution: Combines the outputs of the depthwise convolution using 1×1 convolution.
- This decomposition reduces computational cost from $(O(D^2 \cdot M \cdot N))$ to $(O(D^2 \cdot M + M \cdot N))$, where:
 - D: Kernel size
 - M: Input channels
 - N: Output channels
- MobileNet introduces two hyperparameters:
 - Width Multiplier: Scales the number of channels in each layer to reduce the model size.
 - Resolution Multiplier: Scales the input image resolution to reduce the computation.
- MobileNet models are available pre-trained on ImageNet, making them ideal for transfer learning tasks.
- MobileNet is designed to perform well on devices with limited computational power while maintaining reasonable accuracy.
- MobileNet Variants:
 - MobileNetV1: The first version, introduced in 2017, popularized the use of depthwise separable convolutions.
 - MobileNetV2: Added inverted residual blocks and linear bottlenecks to improve performance and efficiency.
 - MobileNetV3: Uses AutoML to search for better architectures and includes squeeze-and-excitation (SE) modules for better accuracy and efficiency.
- Advantages of MobileNet:
 - Lightweight: Requires fewer parameters and computations compared to traditional CNNs like VGG16 or ResNet.
 - Fast Inference: Optimized for speed, making it suitable for real-time applications.
 - Transfer Learning: Easily fine-tuned for a wide range of tasks using pre-trained weights.
 - Flexible: Can be customized using width and resolution multipliers to suit specific hardware constraints.

In this task we use MobileNetV1 due to these reasons:

- **Lightweight and Efficient:** Ideal for large datasets like handwritten word classification, especially when resources are limited.
- **Pre-Trained Weights:** The pre-trained model on ImageNet can provide a strong starting point, reducing the data and time required for training.
- **Fine-Tuning Capabilities:** Its architecture allows easy fine-tuning, enabling adaptation to specific datasets.

Code Explanation

- **Input:** Path to dataset, image size (224, 224).
- **Processing Steps:**
 - **Grayscale Conversion:** Images are loaded in grayscale for simplicity.
 - **Normalization:** Pixel values are scaled to the range [0, 1] for better convergence during training.
 - **Conversion to RGB:** Since MobileNet expects RGB input, grayscale images are converted to 3 channels.
 - **Label Encoding:** Class names are mapped to numeric indices, and labels are one-hot encoded for multi-class classification.
- **Output:** Preprocessed image arrays, one-hot encoded labels, and class names.
- **Parameters:**
 - `weights='imagenet':` Initializes the model with pre-trained weights.
 - `include_top=False:` Excludes the fully connected layers, as they are replaced with task-specific layers.
 - `input_shape=(224, 224, 3):` Specifies the input size for images.
- The lower layers are frozen to preserve pre-trained features.
- The last 10 layers are unfrozen to allow learning task-specific features.
- **Flatten Layer:** Converts the 3D feature map from MobileNet into a 1D vector suitable for fully connected layers.
- **Dense Layer (256 units):**
 - **Activation:** ReLU introduces non-linearity, enabling the model to learn complex patterns.
 - **Regularization:** `kernel_regularizer=tf.keras.regularizers.l2(0.01)` prevents overfitting by penalizing large weights.
- **Dropout Layer (50%):**
 - Randomly sets 50% of the input units to zero during training to prevent overfitting.
- **Output Layer:**
 - **Number of units:** Equal to the number of classes.
 - **Activation:** softmax outputs a probability distribution for multi-class classification.

- **Optimizer:** Adam optimizer (learning_rate=0.0001) is used for efficient gradient-based optimization.
- **Loss Function:** Categorical cross-entropy measures the difference between predicted and actual label distributions.
- **Data Augmentation Techniques Used:**
 - rotation_range=10: Randomly rotates images up to 10 degrees.
 - width_shift_range=0.1 and height_shift_range=0.1: Shifts images horizontally/vertically by up to 10% of their width/height.
 - zoom_range=0.2: Randomly zooms in/out by 20%.
 - horizontal_flip=False: Ensures that images are not flipped, which may not be applicable for some datasets.
 - fill_mode='nearest': Fills missing pixels after transformations using the nearest neighbor method.
- The model is trained using the augmented dataset with a batch size of 32 over 50 epochs.
- **Metrics:**
 - Test Loss: Measures how well the model performs on unseen data.
 - Test Accuracy: Proportion of correctly classified images in the test set.

Results of task 4:

- **Training Data:** 5700 images across 82 classes.
- **Testing Data:** 2444 images.
- **Training Progress:**
 - **Epochs 1–10:**
 - **Training Accuracy:** Started at 1.58% and gradually improved to 10.46%.
 - **Validation Accuracy:** Showed a similar upward trend, reaching 17.10%.
 - **Observation:** Initial low accuracies indicate the network's early learning phase, where weights are adjusted, and feature extraction begins.
 - **Epochs 11–20:**
 - **Training Accuracy:** Increased from 10.18% to 20.77%.
 - **Validation Accuracy:** Notable improvement to 40.51%, indicating better generalization.
 - **Loss Reduction:** Significant drop in both training and validation loss, with validation loss reaching **2.83** by Epoch 20. This suggests that the model is converging effectively.
 - **Epochs 21–34:**
 - **Training Accuracy:** Continued improvement, reaching **36.51%**.
 - **Validation Accuracy:** peaking at **60.11%** by Epoch 33.
 - **Loss Behavior:** Validation loss decreased consistently to **1.99**, highlighting improved predictive performance.

- **Early Overfitting Mitigation:** The use of data augmentation helped prevent early overfitting by generating diverse training examples.
- **Final Epochs:** By **Epoch 50**, the training accuracy reached approximately **67.23%**, while the validation accuracy peaked at **48.75%**, reflecting significant generalization capability. The training loss decreased to **2.1133**, and the validation loss dropped to **1.7291**, indicating effective learning and minimal overfitting.

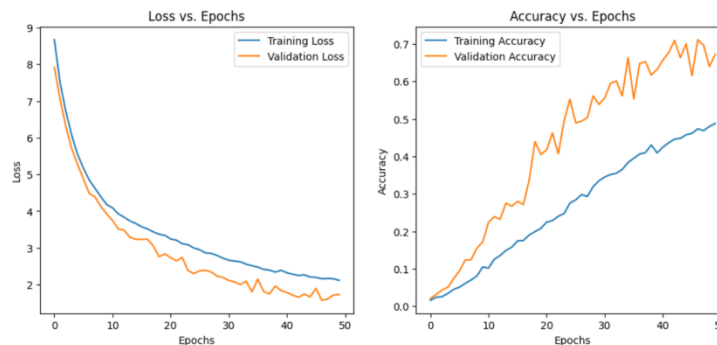


Figure 8 :Loss and Accuracy's vs. Epochs for MobileNetV1

Comparison between the results of task 4 with task 3:

VGG16:

- Training Accuracy: 93.27%
- Test Accuracy: 87.12%
- Training Loss: 0.6035
- Test Loss: 0.8726
- The model demonstrates **high training and validation accuracy**, reflecting strong generalization. The **validation accuracy closely aligns with the training accuracy**, indicating minimal overfitting. Moreover, it **achieves substantially higher accuracy compared to MobileNetV1**, underscoring its effectiveness.

MobileNetV1:

- Training Accuracy: 48.75%
- Test Accuracy: 67.23 %
- Training Loss: 2.1133
- Test Loss: 1.7291
- The training and validation accuracy **show gradual improvement** but remain considerably lower than those of VGG16. The **low training accuracy and trailing validation accuracy** suggest that the architecture may lack the capacity to capture sufficient complexity. While **data augmentation aids generalization slightly**, the **architecture appears to be a significant limiting factor** in achieving better performance.

In the end, if accuracy is the main goal, VGG16 is better than MobileNetV1, especially in applying the right regularization to help it generalize better. But if we need to focus on efficiency, speed, and scalability MobileNetV1 would be the better choice, as it's built to be lightweight and fast without sacrificing too much performance.

Table 2:Comperesion between VGG16 and MobileNetV1

Metric	VGG16	MobileNetV1
Training Accuracy	93.27%	48.75%
Validation Accuracy	87.12%	67.23%
Training Loss	0.6035	2.1133
Validation Loss	0.8726	1.7291
Convergence Rate	Fast	Slow
Overfitting	Minimal	Moderate
Generalization	Strong	Moderate

Conclusion

In conclusion, this project successfully explored the application of deep learning techniques for Arabic Handwritten Text Identification, achieving significant milestones in model performance and generalization. The custom CNN models, particularly Model 1, provided a strong baseline with a test accuracy of 54.99%, while data augmentation further improved its performance to 59.66%, demonstrating the value of enhancing dataset diversity. The fine-tuned VGG16 model emerged as the most effective approach, achieving an impressive test accuracy of 87.12%, highlighting the power of transfer learning and pre-trained architectures for complex tasks. MobileNetV1, though less accurate with a test accuracy of 67.23%, showcased its potential for resource-efficient applications. These findings underscore the importance of model selection, data augmentation, and transfer learning in achieving robust handwritten text recognition systems. For future work, exploring advanced architectures like ResNet or EfficientNet, optimizing hyperparameters, and expanding the dataset to include more diverse handwriting styles could further enhance performance. Additionally, lightweight models like MobileNetV2 or V3 could be investigated for real-time or mobile applications, balancing accuracy and computational efficiency to meet practical deployment needs.